lsp: https://neovim.io/doc/user/lsp.html#LSP
lspconfig: https://github.com/neovim/nvim-lspconfig

LSP (language server protocol)
Wiki: https://en.wikipedia.org/wiki/Language_Server_Protocol
List of LSPs: https://microsoft.github.io/language-server-protocol/implementors/servers/
Protocol for implementing **language specific IDE** features in a code editor.
e.g. go to definition
e.g. code completion
e.g. refactoring
JSON-RPC based
Originally developed by Microsoft for **Microsoft Visual Studio Code** in 2016

neovim + LSP
https://neovim.io/doc/user/lsp.html

lspconfig
LSP only defines **core API** such as functions in **Lua**
e.g. vim.lsp.buf.definition() to find definition
e.g. v:lua.vim.lsp.omnifunc to use for **omnifunc**
lspconfig provides an easier **interface** to setup LSP
e.g. setup connection to **language server**
e.g. setup key **mappings**

But neither lsp nor lspconfig provide **default settings** for any language
These need to be defined manually.
e.g. connect to flow language server for JS
e.g. define key **mappings**
e.g. set **omnicomplete**
e.g. provide **autocomplete** (need to use nvim-cmp)

LSP languages
List of supported languages: https://github.com/neovim/nvim-lspconfig/blob/master/CONFIG.md

Help
:help lsp
:help lspconfig

lspconfig settings
Some configs: https://github.com/neovim/nvim-lspconfig/wiki/Understanding-setup-%7B%7D

:help lspconfig
:help lspconfig-keybindings

:help lsp-client lsp config settings

Mainly 2 tasks:
1) connect to **language server**:
i.e. register language server to lsp so that it **connects** to the server when code is opened.

2) define language **setup**:
e.g. omnifunc
e.g. key mappings.

```
-- In lua
local nvim_lsp = require('lspconfig')

----  Language server settings
-- Needs to be defined to use language server
-- If defined, connects to language server when file is
opened.
-- Some settings: https://github.com/neovim/nvim-
lspconfig/wiki/Understanding-setup-%7B%7D
-- language servers: https://github.com/neovim/nvim-
lspconfig/blob/master/CONFIG.md
--
-- e.g. setup for flow (LSP for JavaScript)
--
-- see
--    :help lsp-client
--
vim.lsp.config("flow", {
 cmd = "npx --no-install flow lsp",        -- cmd:
command to start language server
 filetypes = {"javascript"},              -- filetypes:
which filetypes to enable
 autostart = true,
 root_directory = function () return vim.fn.getcwd() end
 on_attach = …
})
vim.lsp.enable("flow")

vim.lsp.config("flow", { … })        -- setup flow
(JavaScript LSP)
vim.slp.config("bashls", { … })      -- setup bashls
(bash LSP)


-- Multiple setup: overrides previous
vim.lsp.enable("flow")
vim.lsp.config("flow", { … })       -- first setup
vim.lsp.config("flow", { … })       -- second setup.
Overrides first.
```

```
-- [OLD: nvim < 0.11]
--
-- See :help lspconfig-nvim-0.11
--
nvim_lsp.flow.setup {
 cmd = "npx --no-install flow lsp",        -- cmd:
command to start language server
```

```
 filetypes = {"javascript"},              -- filetypes:
which filetypes to enable
 autostart = true,
 root_directory = function () return vim.fn.getcwd() end
 on_attach = …
}

nvim_lsp.flow.setup { … }        -- setup flow
(JavaScript LSP)
nvim_lsp.bashls.setup { … }      -- setup bashls (bash
LSP)


-- Multiple setup: overrides previous
nvim_lsp.flow.setup { … }        -- first setup
nvim_lsp.flow.setup { … }        -- second setup.
Overrides first.
```

LSP Lua API
:help lsp
:help lsp-core

```
-- LSP client object
-- i.e. the connection to the language server
client
client.name
client.id
client.rpc
client.config
…


---- omnifunc
vim.lsp.omnifunc


-- Language functions
vim.lsp.buf.definition()        " Go to definition under
cursor
vim.lsp.buf.declaration()
…

-- Diagnostics
-- i.e. show errors
vim.lsp.diagnostic.show_line_diagnostic() " error
details
vim.lsp.diagnostic.goto_next()            " next error
vim.lsp.diagnostic.goto_prev()            " last error
vim.lsp.diagnostic.set_loclist()          " show in
location list
```

LSP logs
default log path = ~/.cache/nvim/lsp.log

```
" Set log level
lua vim.lsp.set_log_level("debug")

" Get log path
lua vim.cmd('echo "'..vim.lsp.get_log_path()..'"')

" Open log in tab
lua vim.cmd('tabedit'..vim.lsp.get_log_path())


" Write to log
lua vim.lsp.log.debug("abc")
lua vim.lsp.log.info("abc")
lua vim.lsp.log.warn("abc")
lua vim.lsp.log.error("abc")
```

lspconfig Commands

```
" Help
:help lspconfig
:help lspconfig-builtin-commands


:checkhealth vim.lsp      -- previously :nLspInfo
:lsp enable               -- previously :LspStart
:lsp disable
:lsp restart


-- [DEPRECATED nvim <= 0.11]
:LspInfo
:LspStart
:LspStop
:LspRestart
```

==========

nvim-cmp (autocompletion)

nvim-cmp: https://github.com/hrsh7th/nvim-cmp
lspconfig autocompletion wiki: https://github.com/neovim/nvim-lspconfig/wiki/Autocompletion

snippet
source
snippet is the autocomplete UI
i.e. the thing that pops up
snippet is UI engine
e.g. default is vssnipc : https://github.com/hrsh7th/cmp-vsnip

source is what provides the suggestion
i.e. what type of suggestions
e.g. **text source** is just textual i.e. syntactic
e.g. **variable source** is a variable i.e. semantic such as by LSP

sources are provided via **plugins**
e.g. **cmp-treesitter** for syntactic suggestions: https://github.com/ray-x/cmp-treesitter
e.g. **cmp-nvim-lsp** for LSP suggestions: https://github.com/hrsh7th/cmp-nvim-lsp
e.g. **cmp-buffer** for text in same buffer: https://github.com/hrsh7th/cmp-buffer

snippet vs source

